

Departmental Budget Management System

Project Report — ALCHE Programming II Final Project

Princess Nhyira Addai & J'Isabelle Umuvandimwe

Repository: github.com/Pinccissis17/ProgrammingII_Summative

1. Overview

The Departmental Budget Management System is a JavaFX desktop application, backed by a MySQL database through JDBC, that lets a university department administrator record departments in a hierarchy, assign employees to those departments, and track a budget allocation per department per period. The application is built with Maven and follows a layered architecture: a JavaFX UI layer (FXML views and controllers), a model layer (plain Java domain objects), and a data-access layer (DAO interfaces with JDBC implementations) that talks to the database. The three top-level entities are **Department**, **Employee**, and **Budget**; a department can contain any number of employees and any number of sub-departments, which is what motivates both the recursive algorithms and the Composite design pattern described below.

2. Design Patterns

2.1 Composite Pattern

The **OrgUnit** interface (`com.budgetms.model.OrgUnit`) declares `getName()`, `calculateBudget()` and `getHeadcount()`. Both **Department** and **Employee** (`com.budgetms.model.Department / Employee`) implement it. A **Department** holds a list of child **Employee** objects and a list of child **Department** objects, so a department is a composite that can contain both leaf nodes (employees) and other composites (sub-departments), while an employee is always a leaf. This is exactly the intent of the Composite pattern: it lets client code call `calculateBudget()` or `getHeadcount()` on a department without caring whether the total comes from a single employee, a sub-department, or an entire tree of sub-departments — the recursion described in Section 4 is what makes that uniform treatment possible. We chose Composite specifically because the department structure at ALCHE (and at any real institution) is a tree, not a flat list: a School can contain Departments, a Department can contain Units, and every level needs the same two operations rolled up correctly.

2.2 DAO (Data Access Object) / Repository Pattern

Every entity has a DAO interface (`DepartmentDAO`, `EmployeeDAO`, `BudgetDAO`, all in `com.budgetms.dao`) and a JDBC-backed implementation (`DepartmentDAOImpl`, `EmployeeDAOImpl`, `BudgetDAOImpl`) that does the actual SQL. The JavaFX controllers (`com.budgetms.ui.controller.DepartmentController`, `EmployeeController`, `BudgetController`) depend only on the DAO *interfaces*, never on the JDBC classes directly. This separates persistence concerns from UI logic: the controllers describe *what* they need (`create()`, `findAll()`, `update()`, `delete()/deactivate()`) without knowing *how* it is stored, and every SQL statement lives in one place per entity, which is where the tests in Section 5 target their assertions. It also means the storage layer could be swapped (a different database, an in-memory fake for testing) without touching a single controller or FXML file.

3. Data Model

3.1 Entity-Relationship Diagram

Departmental Budget Management System — Entity-Relationship Diagram

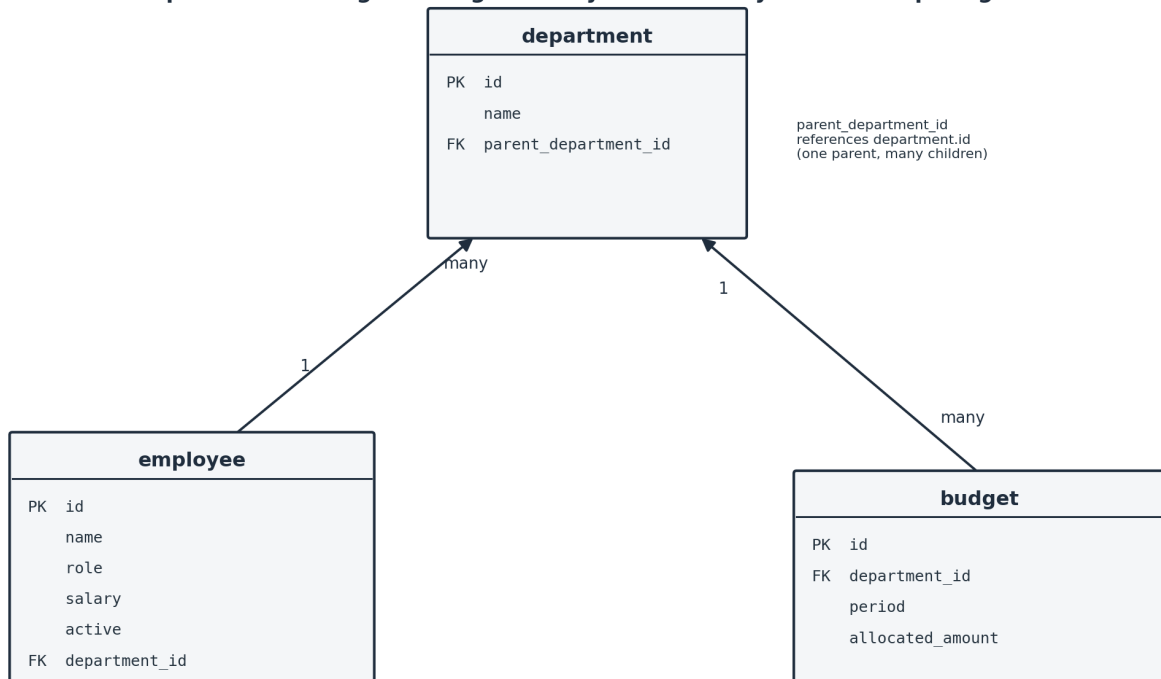


Figure 1. department is self-referencing through parent_department_id (one parent, many children), and is the parent side of one-to-many relationships with both employee and budget.

3.2 Class Diagram

Departmental Budget Management System — Class Diagram

Dashed arrow = implements / realizes. Filled diamond = composition. Hollow diamond = aggregation.

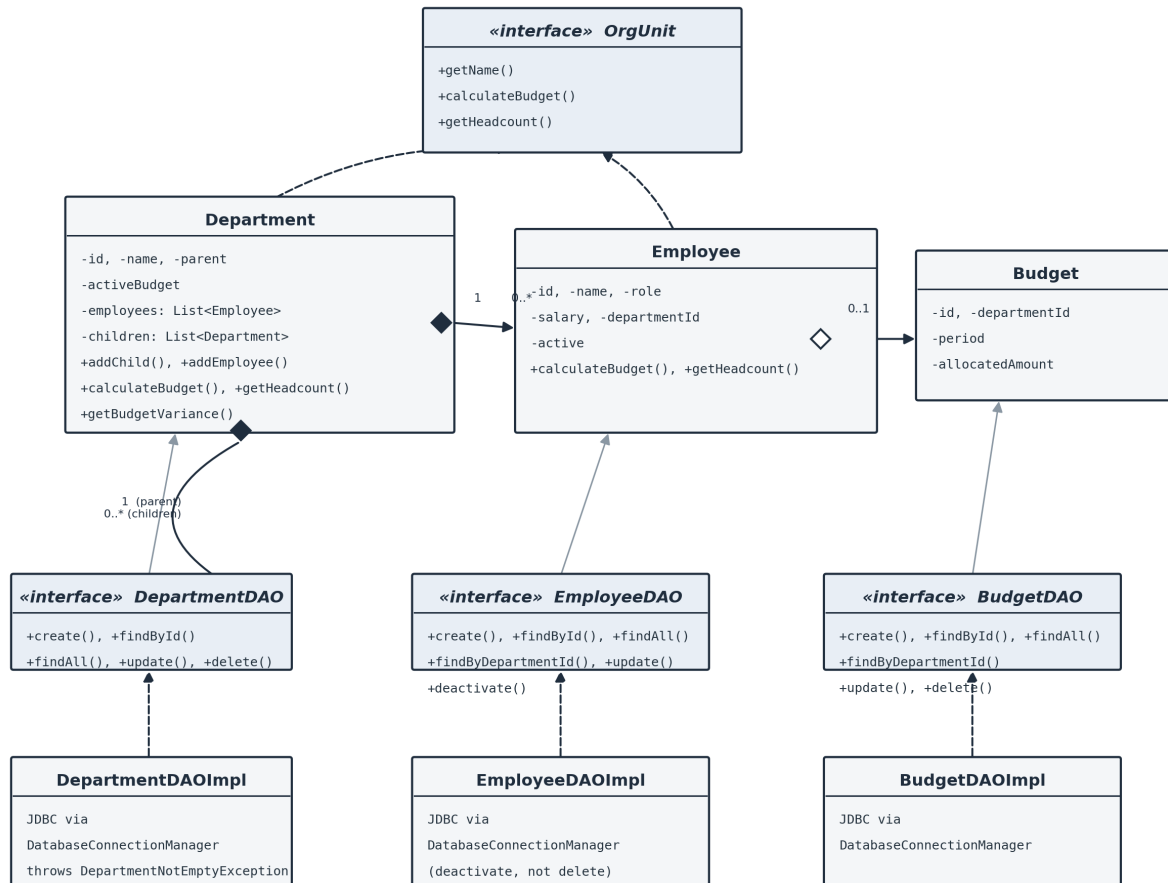


Figure 2. *Department and Employee both implement OrgUnit (Composite pattern). Each DAO interface is implemented by exactly one JDBC-backed class (DAO / Repository pattern).*

4. Recursive Algorithm

Where it lives: `Department.calculateBudget()` and `Department.getHeadcount()`, in `com.budgetms.model.Department`.

Because a department can contain both employees and an arbitrary number of sub-departments, working out a department's total budget or total headcount means walking the whole subtree rooted at that department. Both methods use the same shape of recursion: sum a direct contribution from this department's own employees, then add the same call made on every child department.

```

public double calculateBudget() {
    double total = activeBudget != null ? activeBudget.getAllocatedAmount() : 0.0;
    for (Employee e : employees) {
        total += e.calculateBudget();
    }
    for (Department child : children) {
        total += child.calculateBudget(); // recursive case
    }
    return total;
}

public int getHeadcount() {
    int count = 0;
    for (Employee e : employees) {

```

```

        if (e.isActive()) count++;           // base case contribution
    }
    for (Department child : children) {
        count += child.getHeadcount();       // recursive case
    }
    return count;
}

```

Base case: a department with an empty `children` list stops recursing and returns only what it can compute directly from its own employees list (and its own budget, for `calculateBudget()`). A leaf department (no sub-departments) therefore terminates the recursion immediately. **Recursive case:** for every non-leaf department, the method adds its own direct contribution to the result of calling itself on each child, so the total for the whole tree is built bottom-up from the leaves.

Bounding the recursion: because the tree is built by user input (an administrator adding departments through the UI), nothing stops a careless chain of `parent`→`child`→`child`→`child` from growing very deep or, worse, from being wired into a cycle. `Department.addChild()` guards against this directly: before a department is linked in as a child, it checks the prospective child's depth against a constant `MAX_DEPTH = 6` and throws an `IllegalStateException` if attaching it would exceed that limit. This keeps `calculateBudget()` and `getHeadcount()` provably terminating: recursion depth is capped at 6 regardless of how the tree is built, and the boundary is exercised directly by unit tests (Section 5) that build a 7-level chain and assert the exception is thrown at the point of attachment.

4.1 Recursive vs. Iterative

The same totals could be computed iteratively with an explicit stack or queue (a manual breadth-first or depth-first traversal), pushing each department's children as they are visited instead of letting the call stack track that state. We chose the recursive form for three reasons. First, it mirrors the problem directly: a department's total *is defined in terms of* the same total on its children, so the recursive definition is also the clearest specification of the algorithm — the iterative version would need extra bookkeeping (an explicit collection acting as the stack) to express the same idea. Second, with `MAX_DEPTH = 6` enforced at insertion time, the recursion depth is small and fixed, so the usual objection to recursion (risk of a stack overflow on deep or unbounded input) does not apply here; an iterative version would remove that theoretical risk but buys nothing in practice at this depth, at the cost of noticeably less readable code. Third, the recursive form keeps the traversal logic inside `Department` itself rather than in a separate traversal utility, which keeps the Composite pattern's promise that a client can call one method on the root and get a correct whole-tree answer without knowing anything about how the tree is shaped.

5. Testing

The project has 32 automated tests across 6 JUnit 5 test classes, run through the maven-surefire-plugin (`mvn test`). All 32 currently pass.

Test class	Tests	What it covers
ValidationTest	8	<code>ValidationUtils.requireNonBlank()</code> / <code>requirePositiveNumber()</code> ; <code>Department.addChild()</code> <code>MAX_DEPTH</code> boundary (7-level chain rejected, normal nesting accepted)
DepartmentDaoTest	5	<code>DepartmentDAOImpl</code> CRUD against the database: create, findAll, update, delete, delete-blocked-when-not-empty
EmployeeDaoTest	5	<code>EmployeeDAOImpl</code> CRUD: create, findAll, findByDepartmentId, update, deactivate
BudgetDaoTest	5	<code>BudgetDAOImpl</code> CRUD: create, findByDepartmentId, update, delete, findById
DepartmentRecursionTest	3	<code>Department.getHeadcount()</code> recursion: empty department, mixed active/inactive employees, 3-level nested departments
DepartmentTest	6	<code>Department.calculateBudget()</code> , <code>getBudgetVariance()</code> , and the <code>addChild()</code> <code>MAX_DEPTH</code> boundary

The DAO test classes exercise real database round-trips against the schema created by `SchemaInitializer`, so they double as an integration check that the application's SQL is correct, not just that the Java logic is correct in

isolation.

6. Building and Running

From source (requires JDK 21+ and Maven):

```
mvn clean install      # compiles, runs all 32 tests
mvn javafx:run         # launches the application
```

On first launch, SchemaInitializer creates the department, employee and budget tables automatically if they do not already exist (CREATE TABLE IF NOT EXISTS), against a MySQL instance configured for the project.

Packaged executable: the application can also be built into a self-contained native application with a bundled Java runtime, using jpackage (`--type app-image`), so it runs on a Windows machine that does not have Java installed. The packaged build and the download link/build command are provided separately in the repository README.

7. Team Contributions

Princess Nhyira Addai

Wired all three JavaFX controllers (Department, Employee, Budget) to their DAO interfaces so the UI reads from and writes to the database instead of an in-memory placeholder, including handling domain exceptions raised by the model and DAO layer (IllegalStateException from the MAX_DEPTH check, DepartmentNotEmptyException on delete). Extracted a shared ValidationUtils class from duplicated inline validation in the controllers and wrote ValidationTest (8 tests) covering it and the MAX_DEPTH boundary. Fixed a bug where the employee table reset on navigation instead of reloading from the database. Managed the merge of the DAO-backed branch into main, and built and debugged the native jpackage executable (resolving a missing-module runtime crash via jdeps) so the application runs without a separate Java installation. Authored this report.

J'Isabelle Umuwandimwe

Built the JavaFX UI: the FXML views, styling, and the initial controller and model scaffolding for the application. Wrote the DAO-layer and recursion unit tests (DepartmentDaoTest, EmployeeDaoTest, BudgetDaoTest, DepartmentRecursionTest, DepartmentTest — 24 of the project's 32 tests). Set up the Maven packaging configuration (maven-shade-plugin) used as the basis for the executable build, and is finalizing the packaged build and the project README.

Note to J'Isabelle: this paragraph is a draft based on the git history — please edit it to add or correct anything before we submit.